

How Much Is One Recurrence Worth?

Iso-Depth Scaling Laws for Looped Language Models

Kristian Schwethelm¹Daniel Rückert^{1,2,3}Georgios Kaissis⁴¹Chair for AI in Healthcare and Medicine, Technical University of Munich, Germany²Department of Computing, Imperial College London, UK³Munich Center for Machine Learning (MCML), Germany⁴Hasso Plattner Institute for Digital Engineering, University of Potsdam, Germany

Abstract

We measure how much one recurrence is worth to a looped (depth-recurrent) transformer, in equivalent unique parameters. From an iso-depth pretraining sweep across recurrence counts $r \in \{1, 2, 4, 8\}$ spanning $\sim 50\times$ in training compute, we fit a joint scaling law $L = E + A(N_{\text{once}} + r^\varphi N_{\text{rec}})^{-\alpha} + B D^{-\beta}$ and measure a *recurrence-equivalence exponent* $\varphi = 0.46$. Intuitively, φ tells us whether looping a block r times is equivalent in validation loss to r unique blocks of a non-looped model (full equivalence, $\varphi=1$) or to a single block run repeatedly with no capacity gain ($\varphi=0$). Our $\varphi = 0.46$ sits in between, so replacing unique blocks with shared recurrences *increases* validation loss at matched training compute. For example, at $r=4$ a 410M looped model performs on par with a 580M non-looped model, but incurs the training cost of a 1B non-looped one. We demonstrate the utility of φ as a diagnostic tool on two case studies: commonly used truncated backpropagation lowers φ to 0.38, indicating that the loop mechanism is poorly trained under truncation, even though validation loss decreases. Conversely, hyperconnections raise φ to 0.65, a genuine capacity gain. Our method separates true loop improvements from training-side gains, a distinction raw validation loss cannot make.

1 Introduction

Can a transformer block looped r times replace r non-looped blocks at matched compute budget? Looped, or depth-recurrent, transformers iterate a shared block of layers multiple times [1, 2]. The looped architecture decouples unique parameter count from effective depth, and introduces an inductive bias toward reasoning [3]. These properties have motivated a recent wave of work on looped language models [4–10]. However, in practice, most looped LMs use only small recurrence counts [5–7], revealing a cost: a shared block looped r times may not fully substitute for r unique blocks. Intuitively, reusing a block leads to higher compute cost (FLOPs) per parameter, so under a limited FLOPs budget, the number of unique blocks must be reduced. The shared parameters must then do more work to be worth the trade. How many unique parameters one recurrence is worth has not been measured directly. Concurrent scaling-law work [10] fixes the unique parameter count, identifying compute-optimal recurrence settings, but parameter sharing, effective depth, and inference cost vary together in their setup, so the per-parameter value of a recurrence cannot be isolated.

To isolate parameter sharing from effective depth, we run an iso-depth scaling sweep across four prelude-recur-coda architectures with recurrence count $r \in \{1, 2, 4, 8\}$, where $r=1$ is the non-looped baseline (see schematic in Appendix Figure 5). By design, the four variants execute the same number of forward layers per token and, at matched width d_{model} , incur the same per-token training and inference FLOPs. Yet unique non-embedding parameters drop by $3.2\times$ as r grows (Figure 1, left), the *parameter-sharing cost* we quantify. We sweep six compute budgets from 4.64×10^{17} to 2.15×10^{19} FLOPs ($\sim 50\times$) to find the compute-optimum per architecture, yielding 116 runs.

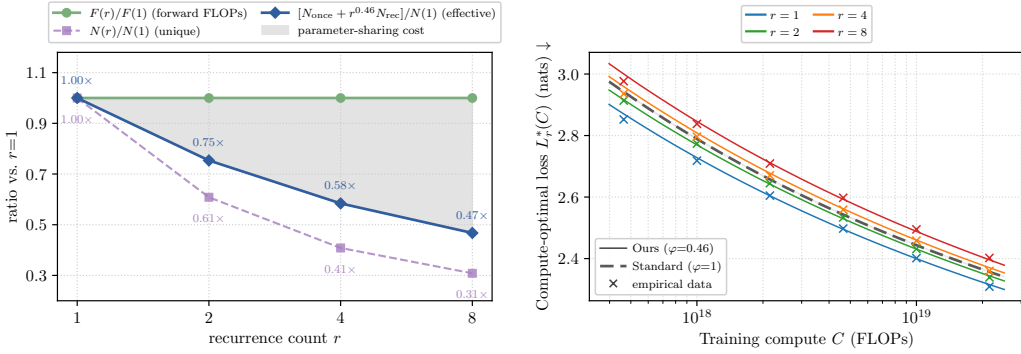


Figure 1: **How much is one recurrence worth?** *Left*: at matched effective depth, per-token forward FLOPs $F(r)$ stay flat while unique parameters $N(r)$ drop as r grows. Effective parameters $N_{\text{once}} + r^\varphi N_{\text{rec}}$ with $\varphi=0.46$ drop more slowly. *Right*: compute-optimal validation loss per architecture against compute budget C . Empirical per-budget optima (crosses) track our $\varphi=0.46$ fit (solid curves). The standard form ($\varphi=1$) collapses all four architectures onto a single curve (dashed), which fits none of the empirical results.

On these runs, the standard Chinchilla scaling law $L(N, D)$ [11] can relate validation loss to unique parameters N and training tokens D , but has limited utility in comparing looped models. The recurrence count r does not enter the law, and the parameter count N has different FLOPs cost under looping. We therefore formulate a joint scaling law $L(N_{\text{once}}, N_{\text{rec}}, D, r)$ that includes r and splits N into the shared recurrent block (N_{rec}) and the single-use prelude and coda (N_{once}), with $N_{\text{once}} + N_{\text{rec}} = N$. Fully-looped models are recovered by $N_{\text{once}} = 0$.

At the core of our joint scaling law, we introduce a *recurrence-equivalence exponent* φ that quantifies how much each shared parameter contributes to loss reduction relative to a unique one: $L(N_{\text{once}}, N_{\text{rec}}, D, r) = E + A(N_{\text{once}} + r^\varphi N_{\text{rec}})^{-\alpha} + B D^{-\beta}$. φ has two natural reference points. $\varphi = 1$ attributes full parameter equivalence to each recurrence (no sharing cost), so at matched training FLOPs the looped model would reach the same validation loss as the non-looped baseline. $\varphi = 0$ indicates no gain through recurrences (a pure sharing cost). For our baseline architecture, we find $\varphi = 0.46$ (Figure 1). This means a 410M $r=4$ model performs on par with a 580M non-looped model, but incurs the training cost of a 1B non-looped one (derivation in Appendix F.4).

Is φ fixed? No. We propose the shift $\Delta\varphi$ as a diagnostic tool and demonstrate it on two case studies: truncated backpropagation [4, 8, 10] (gradients are only computed for the last few loops, saving $\sim 30\%$ training FLOPs) and hyperconnections [12] (parallel residual streams between loops). Both decrease validation loss but move φ in opposite directions. Truncated backpropagation decreases φ to 0.38, which means each loop is *less* powerful. This is offset by lower training cost, allowing increases in parameter count (model width) and training tokens. However, more parameters result in higher inference cost. Each loop therefore produces less capacity per FLOP, wasting test-time compute. In contrast, hyperconnections raise φ to 0.65 and lower inference cost. $\Delta\varphi$ thus distinguishes whether a validation-loss gain comes from a better loop mechanism or a hidden trade-off with inference cost, a distinction raw validation loss cannot make.

Contributions.

1. We introduce a *recurrence-equivalence exponent* φ that quantifies, via a joint scaling law over r , how many unique parameter blocks one recurrence is worth, and measure a baseline of $\varphi = 0.46$: each recurrence is worth roughly half a unique block at the same FLOPs.
2. We propose $\Delta\varphi$ as a diagnostic tool for comparing looped LM training recipes and architectures. It separates architecture-side from training-side gains, with the compute-optimal allocation reflecting the resulting deployment cost.
3. By measuring $\Delta\varphi$, we show that commonly used truncated backpropagation weakens looping and raises inference cost ($\varphi = 0.38$), while hyperconnections provide a genuine capacity gain that also narrows the compute-optimum ($\varphi = 0.65$).

2 Related Work

Looped language models. The Universal Transformer [1] introduced weight sharing across depth. Such looped models have recently drawn renewed attention in language modelling as a route to implicit, latent-space reasoning and test-time compute scaling, where iterating a shared block lets a model spend more compute per token. Huginn [4] and Ouro [5] have scaled the paradigm to $\sim 3\text{B}$ parameters and trillion-token training budgets with strong downstream results, often matching much larger dense transformers. However, this comes at a proportional training-compute cost, as each recurrence multiplies forward-backward FLOPs. Another line of work runs compute-matched comparisons at single training budgets and reports a consistent pattern: looped models trail non-looped baselines on validation loss and parametric-knowledge tasks but close the gap or outperform them on reasoning benchmarks [3]. We extend these findings to the scaling-law setting [11]. Unlike per-architecture scaling laws, our joint law fits looped and non-looped models together under a single recurrence-equivalence exponent φ . Architectural and training-efficiency methods such as retrofitting [8, 9], adaptive compute [6, 7, 13, 14], truncated backpropagation [4, 8, 10], and hyperconnections [12, 15] affect φ . We apply our framework to truncated backpropagation and hyperconnections, leaving the others to future work. See Appendix A for extended discussion.

Iso-parameter scaling laws. Concurrent work by Prairie et al. [10] fits a scaling law at fixed unique parameter count, motivated by equal parameter memory footprint between architectures. However, in contrast to our setup, depth, per-token inference FLOPs, and KV cache memory all grow with the recurrence count. The two setups therefore answer different questions: Prairie et al. [10] trace the compute-optimal recurrence count under a limited memory budget, while we measure the per-recurrence sharing cost by fixing effective depth. See Appendix A for a detailed comparison.

3 Methodology

We compare four transformer variants: a non-looped baseline ($r=1$) and looped models with $r \in \{2, 4, 8\}$ recurrences, all with 20 effective layers (iso-depth). At same model width, per-token training and inference FLOPs match across r up to a small correction for an input-injection layer, but the unique parameter count shrinks substantially as r grows.

3.1 Looped Transformer Architecture

All looped variants follow the prelude-recur-coda template [4] with fixed effective depth $\ell_{\text{eff}} = \ell_{\text{prelude}} + r \cdot \ell_{\text{recur}} + \ell_{\text{coda}} = 20$. We set $(\ell_{\text{prelude}}, \ell_{\text{coda}}) = (2, 2)$, which yields a shared recurrent block of $\ell_{\text{recur}} = 16/r$ layers executed r times, giving $(8, 4, 2)$ recurrent layers for $r \in \{2, 4, 8\}$. Width is parameterised as $d_{\text{model}} = 64s$ for an integer scale factor s , with attention head dimension 128.

We follow Geiping et al. [4] and use a linear input-injection layer (Appendix C.2). Its small FLOPs overhead is included in every iso-FLOPs comparison. Full architectural details are in Appendix C.

3.2 FLOPs Accounting Under Parameter Sharing

Let $n_b = 12d^2$ be the parameter count of a single transformer layer at width $d \equiv d_{\text{model}}$ (four $d \times d$ attention projections plus the $d \rightarrow 4d \rightarrow d$ MLP), and $n_i = 2d^2$ that of the injection layer. We write N for the transformer’s total non-embedding parameters and adopt the standard convention that per-token forward FLOPs $\approx 2N$ and training FLOPs $\approx 6N$ [11, 16]. Our reported FLOPs also include parameter-free attention matmuls ($12hqT$ at training, with h heads of dimension q and sequence length T) and the unembedding matmul ($6dV$ at training, with vocabulary V), both fixed across architectures at matched width. For simplicity we exclude them from the equations below.

For looped models, we split N into parameters that are used once, $N_{\text{once}} = (\ell_{\text{prelude}} + \ell_{\text{coda}})n_b$, and those that are recurring, $N_{\text{rec}} = \ell_{\text{recur}}n_b + n_i$ (fully-looped architectures correspond to $N_{\text{once}} = 0$). In our setup $\ell_{\text{recur}} = 16/r$, so each additional recurrence shrinks the recur block. Thus, our looped models with $r \in \{2, 4, 8\}$ have $\sim 61\%$, $\sim 41\%$, and $\sim 31\%$ of the parameters of a same-width non-looped model.¹ For example, at $s=10$, $N \in \{98.3, 59.8, 40.2, 30.3\}$ M.

¹ $N(r) = (4 + 16/r)n_b + n_i = (48 + 192/r + 2)d^2$, giving $\{146, 98, 74\}d^2$ for $r \in \{2, 4, 8\}$ versus $240d^2$ at $r=1$.

In terms of compute, the looped transformer executes the prelude once, the recurrent block r times (each preceded by the injection layer), and the coda once, so per-token forward FLOPs are:

$$F_{\text{fwd}}(r) = 2(N_{\text{once}} + r N_{\text{rec}}) \approx F_{\text{fwd}}(1) = 2 \ell_{\text{eff}} n_b = 2N. \quad (1)$$

Thus, all variants use the same F_{fwd} up to a small injection layer overhead of $r/120 \in \{1.7\%, 3.3\%, 6.7\%\}$ at $r \in \{2, 4, 8\}$.² Training FLOPs are $F_{\text{train}}(r) = 3 F_{\text{fwd}}(r)$, so at a fixed compute budget C every variant trains on approximately the same token count $D \approx C/F_{\text{train}}$.

Overall, looped models are much smaller, as they spend much more compute per parameter.

3.3 Joint Scaling Law

The standard Chinchilla scaling law is defined as

$$L(N, D) = E + AN^{-\alpha} + BD^{-\beta}, \quad (2)$$

where L is validation loss (nats), N the non-embedding parameter count, D training tokens, E the irreducible loss, and A, B, α, β fitted constants. Loss decreases with higher N and D , approaching E as they grow large. The two terms split the loss into a parameter contribution $AN^{-\alpha}$ and a token contribution $BD^{-\beta}$, with α, β setting how fast each diminishes and (via their ratio) the compute-optimal allocation between N and D [11]. Our iso-compute design holds D approximately fixed across r but reduces N , so the same token count is spread over fewer parameters as r grows.

To isolate how much the looped parameters N_{rec} effectively contribute per recurrence, we extend the Chinchilla form (Equation 2) with a *recurrence-equivalence exponent* $\varphi \geq 0$ acting on N_{rec} :

$$L(N_{\text{once}}, N_{\text{rec}}, D, r) = E + A (N_{\text{once}} + r^\varphi N_{\text{rec}})^{-\alpha} + B D^{-\beta}. \quad (3)$$

We refer to $N_{\text{eff}} \equiv N_{\text{once}} + r^\varphi N_{\text{rec}}$ as the *effective parameter count* throughout. The factor $r^\varphi \geq 1$ amplifies the contribution of the looped parameters to loss reduction.

Intuitively, N_{eff} is the parameter count of a non-looped model that would match the looped variant’s loss at the same D . So, if looped models outperform non-looped models at same N , we attribute the performance gain to looped parameters contributing more than they would in a single forward pass. Note that the law is fit on iso-compute runs, so C enters only implicitly through the data. The non-looped reference matches loss at the same D , not at the same compute. When $\varphi < 1$, the recurrent block’s contribution to capacity grows slower with r than its contribution to compute (Equation 1), so looped variants underperform non-looped models in compute-matched comparisons.

φ has two natural reference points. $\varphi = 0$ means the recurrent block contributes the same to loss whether run once or r times. The extra $r-1$ executions add FLOPs without any capacity gain, and increasing r at fixed compute only shrinks N and thus raises loss. $\varphi = 1$ corresponds to the iso-FLOPs non-looped model, where each recurrence contributes as much as equivalent unique parameter blocks, so all four architectures would perform the same. Values $0 < \varphi < 1$ quantify partial recovery. $\varphi > 1$ would require the looped block to contribute more than unique blocks (eg, $r=8$ outperforms $r=4$), which we do not observe.

4 Iso-Depth Scaling Laws

We train each of the four architectures at six compute budgets, $C \in \{4.64 \times 10^{17}, 1.00 \times 10^{18}, 2.15 \times 10^{18}, 4.64 \times 10^{18}, 1.00 \times 10^{19}, 2.15 \times 10^{19}\}$ FLOPs, sweeping model width d_{model} at each budget to find the compute-optimal point. On the resulting 116 pretraining runs, we first fit per-architecture Chinchilla laws, then our proposed joint law with φ .

4.1 Experimental Details

Implementation. Our implementation builds on nanochat [17]: a decoder-only transformer with RMSNorm [18], RoPE [19], QK normalisation [20], and squared-ReLU MLPs [21]. We use FlashAttention-2 & 3 [22, 23] as the attention backends.

²The overhead is $2rn_i/(2\ell_{\text{eff}}n_b) = r \cdot 2d^2/(20 \cdot 12d^2) = r/120$.

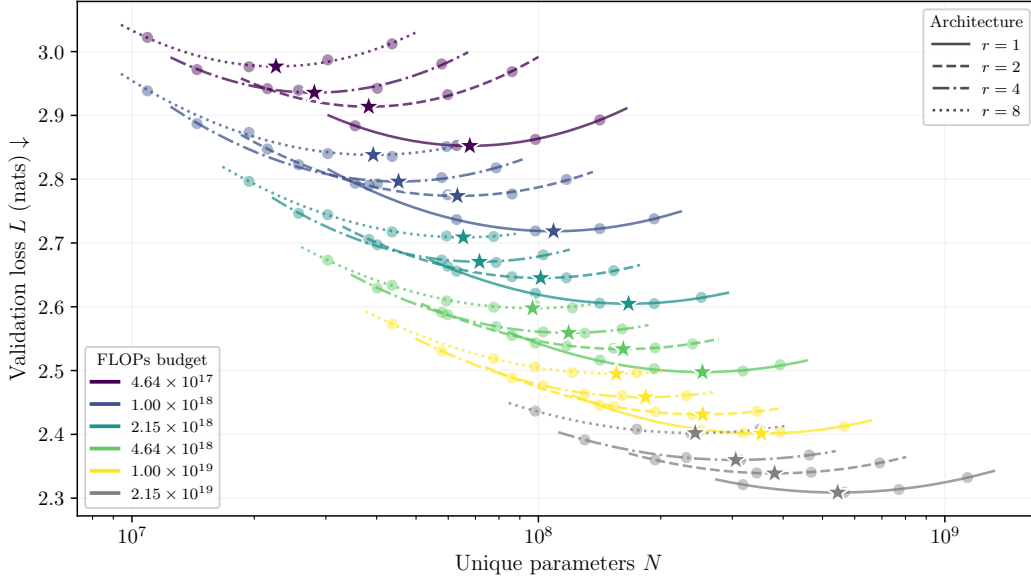


Figure 2: Scaling curves at fixed compute budgets. Thin curves are per-(budget, r) parabolic fits in $\log N$. Stars mark the fitted compute-optimal (N^* , L^*) points.

Optimisation. Matrix parameters are optimised with MuonH [24, 25]. Embedding, unembedding, and norm parameters are optimised with AdamW [26]. Weight decay is set to zero (first-order no-op under MuonH’s Frobenius-sphere constraint [27]). Learning rates transfer across width, batch size, and training horizon via muP [28] and HyperP [27]. In Appendix D, we sweep base LR and batch size at the reference width and find optima agreeing across architectures (LR regret below 0.005 nats per architecture). All runs use base MuonH LR $\eta_{\text{base}} = 0.014$ and AdamW base LR 0.3 (embedding), 0.004 (unembedding), and 0.005 (norm), with each LR linearly decayed to 10% of its peak.

Data and validation. Training data is a subset of FineWeb-Edu [29], tokenised with the Llama 2 tokenizer [30] (32,000 base vocabulary) and pre-packed into fixed-length sequences of 2,049 tokens (the extra token provides the next-token target for position 2,048). Thus, all four architectures see exactly the same data stream. Validation loss is reported in nats on a held-out FineWeb-Edu split packed identically to training.

4.2 Per-Architecture Chinchilla Fits

Figure 2 shows validation loss against unique non-embedding parameters N at our (budget, architecture) grid. At fixed compute C all four architectures trace an approximately parabolic iso-FLOPs curve in $\log N$. The parabolas are systematically offset upward and flatter for larger r , with looped minima at *wider* widths than the non-looped baseline. We fit the standard Chinchilla scaling law (Equation 2) separately for each architecture to characterise its individual scaling behaviour.

Fitting protocol. We follow Hoffmann et al. [11] and minimise the Huber loss [31] ($\delta = 10^{-3}$) between predicted and empirical $\log$ validation loss using L-BFGS [32]. Specifically, we parameterise the law in log-space ($a = \log A$, $b = \log B$, $e = \log E$) and minimise

$$\mathcal{L}(a, \alpha, b, \beta, e) = \sum_i \text{Huber}_\delta(\text{LSE}(a - \alpha \log N_i, b - \beta \log D_i, e) - \log L_i), \quad (4)$$

where LSE is log-sum-exp. The log-space objective treats relative errors uniformly across the wide dynamic range of N and D . Because the objective is non-convex, we take the best of 500 random L-BFGS-B restarts (details in Appendix F). Table 1 reports the fitted parameters per architecture, all achieving $R^2 > 0.997$ (predicted-vs-actual scatter and residuals in Appendix F).

Table 1: Chinchilla scaling-law fit parameters per architecture. Huber loss is the objective at the optimum (Equation 4). R^2 is on raw nats. Amplitudes A, B are rounded to 2 significant figures because they are only loosely identified under iso-compute designs [33].

Arch.	A	α	B	β	E	Huber ($\times 10^{-5}$)	R^2
$r=1$	58	0.251	150	0.267	1.56	5.84	0.9979
$r=2$	33	0.216	910	0.365	1.60	5.27	0.9983
$r=4$	23	0.191	1300	0.388	1.56	6.81	0.9976
$r=8$	41	0.235	780	0.362	1.69	5.33	0.9980

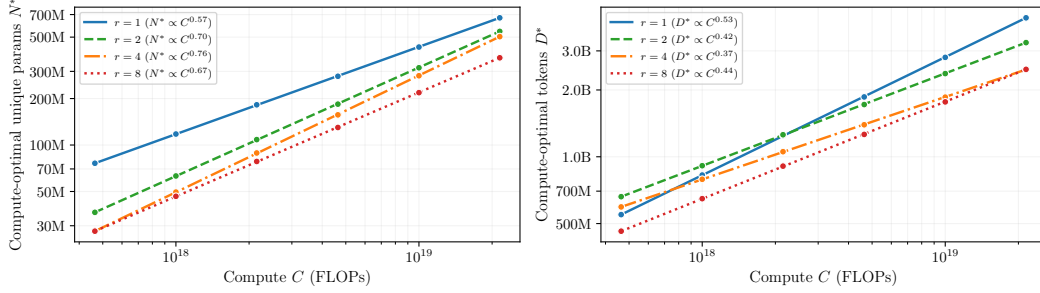


Figure 3: Compute-optimal allocation per architecture. Left: optimal unique parameter count $N^*(C)$ with fitted exponent in the legend. Right: optimal training tokens $D^*(C)$.

Compute-optimal allocation and gap. The fitted exponents in Table 1 characterise each architecture’s scaling behaviour in isolation but are not directly comparable across r . The unique parameter count N has different semantics under parameter sharing (each parameter in the recurrent block contributes r times for looped models), and iso-FLOPs sampling places each architecture in a different region of the (N, D) plane, where α and β are weakly identified [33]. The compute-optimal loss frontier $L_r^*(C)$, by contrast, is directly comparable across r . We derive the optimal parameter and token allocation $N^*(C)$, $D^*(C)$ by minimising $L(N, C/F(N))$ over N at each budget, with $F(N)$ the architecture’s empirical per-token FLOPs at N . The results are shown in Figure 3. Looped optima favour wider models than the non-looped baseline, but their unique parameter count is still lower (left). The optimum compensates for parameter sharing by widening, which raises per-token FLOPs and therefore lowers the optimal training-token count at fixed compute (right).

The resulting loss frontier trails the baseline by $[0.03, 0.06]$ nats at $r=2$, $[0.05, 0.08]$ nats at $r=4$, and $[0.09, 0.12]$ nats at $r=8$, growing monotonically with r across the six budgets. The gap widens at lower budgets and flattens at our largest ($\Delta \leq 0.006$ nats between 10^{19} and 2.15×10^{19} FLOPs).

Extrapolation beyond the grid. To test whether the gap persists past our grid, we train $r=1$ and $r=4$ models at $s=34$ on 47B tokens ($\sim 4 \times 10^{20}$ FLOPs, $\sim 20\times$ the top of our grid). The looped model trails by 0.061 nats in validation loss, inside the $[0.05, 0.08]$ nats $r=4$ band measured across the grid.³ Full numbers and protocol are reported in Appendix I.

4.3 Joint Scaling Law Fit φ

We now fit the joint law of Equation 3 across all 116 runs. The recurrence-equivalence exponent φ places every architecture on a common $(N_{\text{once}} + r^\varphi N_{\text{rec}}, D)$ surface and measures how much each recurrence amplifies the contribution of N_{rec} . This also reduces the four per-architecture fits above (20 parameters) to one shared law with 6 parameters ($A, \alpha, B, \beta, E, \varphi$). We minimise the same Huber-on-log objective as Equation 4. The results are shown in Table 2.

We measure $\varphi = 0.46$, well below the non-looped reference of $\varphi = 1$ but clearly above 0. Each recurrence amplifies the looped parameters’ contribution to loss reduction by a factor of $r^{0.46}$, so

³Under iso-token training, the looped model uses $\sim 3\%$ more training FLOPs (from the injection-layer overhead), but $s=34$ also sits further from the $r=4$ compute-optimum than from the $r=1$ optimum, since looped optima favour wider models. The reported gap therefore mixes a small FLOPs advantage for the looped model with a width sub-optimality penalty.

Table 2: Joint $(N_{\text{once}}, N_{\text{rec}}, D, r)$ scaling law (Equation 3) fit. The free- φ row reports 95% block-bootstrap CIs (200 resamples of (budget, architecture) cells) below the φ point estimate. Amplitudes A, B are only loosely identified under iso-compute designs [33] and are omitted from the table.

Form	α	β	E	φ	R^2
Joint, φ free	0.199	0.369	1.57	0.459 [0.41, 0.53]	0.9972
Restricted ($\varphi = 0$, pure sharing cost)	0.227	0.390	1.71	0.00	0.9858
Restricted ($\varphi = 1$, non-looped equivalence)	0.218	0.410	1.66	1.00	0.9552

the recurrent block contributes $\sim 1.9\times$ its unique parameter count at $r=4$ and $\sim 2.6\times$ at $r=8$. This matches the qualitative pattern from the compute-optimal allocation above. Each recurrence increases the capacity of the looped parameters ($\varphi > 0$), requiring fewer tokens to reach equal performance at same N . That is why the optimum can spend a smaller share of its FLOPs on tokens and more on widening the model. However, the added capacity is only partial ($\varphi < 1$): the $r^{0.46}$ amplification of the looped block’s contribution does not keep up with the linear r increase in its FLOPs cost. Per-FLOP capacity therefore falls with r , and the looped frontier trails the non-looped baseline.

Robustness. The 95% block-bootstrap CI on φ is [0.41, 0.53], with no resample reaching $\varphi = 0$ or $\varphi = 1$. The R^2 values in Table 2 look uniformly high because most variance across the runs comes from the compute-budget axis (cross-architecture loss spans only ~ 0.1 nats). On that scale, small drops from 0.997 (free φ) are substantial. Figure 1 (right) visualises this fit-quality gap for $\varphi = 1$. Per- r residuals (Appendix Table 7) are comparable across architectures (RMSE 0.009–0.011 nats). Refits on the low-budget half ($C \leq 2.15 \times 10^{18}$) and high-budget half ($C \geq 4.64 \times 10^{18}$) give $\varphi = 0.44$ and $\varphi = 0.49$, both inside the CI. Full analysis in Appendix F.

5 Case Studies

The fitted $\varphi = 0.46$ describes our baseline recipe and architecture. We measure φ under two candidate interventions: truncated backpropagation (a training-recipe change) and hyperconnections (an architectural change). For each case study we rerun the iso-FLOPs grid for $r \in \{2, 4, 8\}$ at our four lower budgets (4.64×10^{17} to 4.64×10^{18} FLOPs), reuse the unchanged $r=1$ runs as the baseline, and refit the joint law (Table 3). The resulting $\Delta\varphi$ is a single-number summary of how much each recurrence gains or loses capacity under the method. Implementation details are in Appendix G.

5.1 Truncated Backpropagation

Truncated backpropagation through time (BPTT) is a training-efficiency method that is commonly applied to the recursion steps of looped transformers [4, 8, 10]. Under full BPTT, gradients flow backward through all r recurrences. Truncated BPTT detaches the recurrent state for all but the last r_{bwd} loops, so the early recurrences run forward only and skip the backward pass. We follow Prairie et al. [10] and set $r_{\text{bwd}} = \lceil r/2 \rceil$. In our setup, the skipped backward passes save roughly 30% of the per-token training FLOPs, allowing more training tokens at fixed budget.

The scaling curves in Figure 4 (left) show that truncated BPTT substantially lowers validation loss across all runs, suggesting at first glance that the method works well. However, the measured φ disagrees, dropping from 0.45 to 0.38 (Table 3). So each additional recurrence is now worth much less in unique-parameter terms than before (smaller r^φ in N_{eff} , so matching the same N_{eff} now requires a larger N_{rec}). This likely reflects early loops no longer receiving an accurate learning signal. Evidence for this is the $r=2$ architecture, which consistently has the largest residual error (Figure 10). Here $r_{\text{bwd}}=1$, so the shared block receives a direct gradient only from the second recurrence. The first recurrence still runs forward and conditions the second loop’s input, but this indirect signal is evidently too weak to train the looping mechanism. The same failure applies to the forward-only loops at $r=4$ and $r=8$, which lowers the per-recurrence utility and pulls φ down. Refitting on $r \in \{4, 8\}$ alone leaves φ essentially unchanged at 0.37, so the capacity drop is architecture-spanning rather than an $r=2$ artifact. Overall, the joint law attributes the validation-loss improvement to a larger token budget and a wider compute-optimal model that offsets the per-loop capacity loss but raises per-token inference cost.

Table 3: Joint-law refits under each case study alongside the full-BPTT, linear-injection baseline. Baseline was refitted to the four lower budgets. All fits use the same Huber-on-log objective and bounds.

Variant	α	β	E	φ	R^2
Baseline (full BPTT, linear injection)	0.266	0.457	1.85	0.453	0.9959
Truncated BPTT (with $r=2$)	0.255	0.484	1.87	0.380	0.9827
Truncated BPTT (without $r=2$)	0.265	0.492	1.89	0.373	0.9958
Hyperconnections	0.308	0.464	1.93	0.646	0.9900

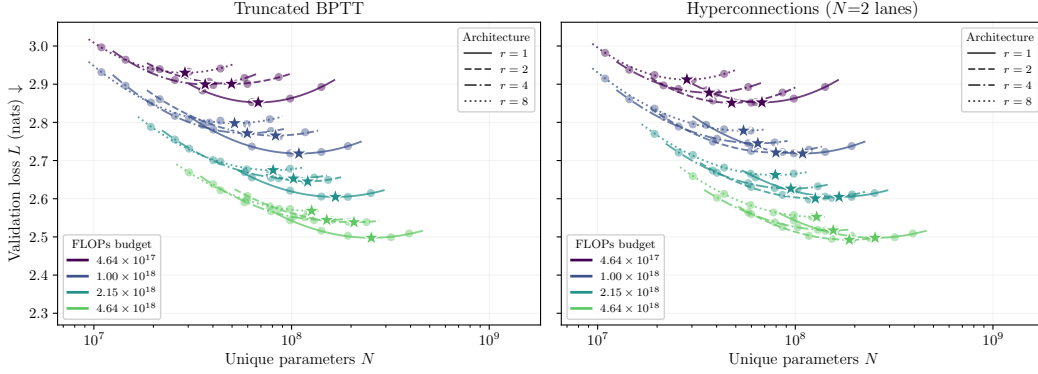


Figure 4: Scaling curves under the two case studies. Thin curves are per-(budget, r) parabolic fits in $\log N$. Stars mark the fitted compute-optimal (N^* , L^*) points.

5.2 Hyperconnections

We replace the linear input injection of our baseline looped model with hyperconnections [12], which scale and mix K parallel residual lanes across loops. We hypothesize that better information flow inside the recurrent block leads to a better loop mechanism. We use $K=2$ lanes and full BPTT.

The scaling curves in Figure 4 (right) show that hyperconnections substantially lower validation loss across all looped runs, and φ jumps from 0.45 to 0.65 (Table 3). Each additional recurrence is now worth much more in unique-parameter terms than before. Interestingly, the $r=2$ architecture even matches or beats the $r=1$ baseline at some budgets. Note that $\varphi = 1$ would require all four architectures to lie on the same compute-optimal frontier, not just $r=2$ beating $r=1$ at a few budgets.

Hyperconnections are a genuine architectural improvement, as validation loss falls and the compute-optimal allocation moves to narrower widths, lowering per-token inference FLOPs (the opposite of the widening seen under truncated BPTT). However, hyperconnections were originally proposed as a replacement for the residual connections between transformer blocks [12] and could in principle be applied to the $r=1$ baseline as well. Our case study applies them only at the loop boundary, which is the natural site for a within-loop intervention. An $r=1$ baseline modified the same way would likely shift the calibration of φ downward. However, our primary comparisons are between looped models, with the non-looped baseline serving only as a calibration anchor. The shift would also likely be modest: concurrent work [15] shows loop-level hyperconnections beat layer-level hyperconnections.

6 Discussion

Per-recurrence value. Our $\varphi = 0.46$ quantifies how much one recurrence is worth in equivalent unique parameters. At $r=4$ the shared block recovers $4^{0.46} \approx 1.89$ unique blocks of capacity, about 47% of full equivalence ($\varphi=1$). The introduction asks whether a block looped r times can replace r non-looped blocks at matched compute. Our measurement says no, not under our recipe. But φ is not a property of looping in general. It reflects the joint state of architecture, optimiser, and training recipe, and the case studies of Section 5 show φ can move substantially under either type of change.

Downstream validation. Our five-axis downstream evaluation in Appendix H is consistent with this view. Parametric-knowledge tasks track validation loss directly, while reasoning-heavy tasks on which looped models are predicted to excel show no above-noise architectural signal at our budgets, not even at the $\sim 20\times$ extrapolation runs. The link between φ and reasoning quality at scale is therefore empirically untested. Validation loss is reliable at development-scale compute, and $\Delta\varphi$ is a useful summary of architectural progress. A reasoning-heavy pretraining mix [13] might surface architectural differences on reasoning tasks at our budgets and offer a specialised $\Delta\varphi$ axis.

$\Delta\varphi$ as a development metric. $\Delta\varphi$ separates token-side from architecture-side gains, a comparison raw validation loss cannot make. A pure training-efficiency method can lower the loss simply by trading compute for more tokens at fixed budget. A pure architectural change can lower the loss by raising per-recurrence capacity. Methods can also combine the two, reducing per-token training FLOPs while raising φ . Our two case studies illustrate the pure cases at either end. Truncated BPTT lowers loss on the token channel but drops φ , while hyperconnections lower loss on the capacity channel by raising φ . We therefore recommend $\Delta\varphi$ as a diagnostic tool alongside validation loss for any new looped LM recipe or architecture. Measuring it is relatively cheap. Per architecture, a focused case study of ~ 20 runs across our four lower budgets totals $\sim 5 \times 10^{19}$ FLOPs, an order of magnitude below our $s=34$ extrapolation run ($\sim 4 \times 10^{20}$ FLOPs). The same runs also yield the per-architecture compute-optimal allocation as a by-product.

Other methods worth quantifying include shrinking the shared fraction (larger prelude/coda), per-token adaptive compute [6, 7, 13, 14], retrofitting pretrained non-looped models [8, 9], and training with a diffusion objective in place of unrolling the loops [34].

Inference cost. φ also predicts inference cost. A higher φ means each loop adds more capacity, so the same loss can be reached with fewer unique parameters. The freed compute then goes into more training tokens, giving a narrower compute-optimum and lower per-token inference FLOPs. A lower φ has the opposite effect. Each loop adds less, so the optimum widens to compensate and trains on fewer tokens, raising inference cost. A method that substantially lowers φ should therefore be treated cautiously, since the same validation loss is reachable more cheaply with fewer, more powerful recurrences. This is exactly what we observe for truncated BPTT (Section 5.1). On the loop-efficiency dimension, φ thus already reflects inference cost.

Limitations. We fix a single architecture configuration: 20 effective layers with $(\ell_{\text{prelude}}, \ell_{\text{coda}}) = (2, 2)$ following the prelude-recur-coda template of Geiping et al. [4]. Different depth allocations or prelude/coda sizes may shift φ , which we leave to future work. Our iso-depth setup also caps recurrences at $r_{\text{max}} = 16$. The r^φ form is a pre-saturation local approximation valid in this range and does not include the architectural ceiling. The joint law also assumes that each additional recurrence must either consistently help ($\varphi > 1$) or consistently hurt ($\varphi < 1$) compared to the non-looped baseline. When some r outperform the baseline and others fall below it, no single φ captures both directions and the fit degrades. The non-looped baseline is therefore an important calibration anchor for φ . Thus, we tune all four architectures with the same recipe in Section 4.1, with Appendix D confirming LR optima agree. Additionally, φ obtained with different baselines are not comparable.

7 Conclusion

We measure the parameter-sharing cost of looped language models as a single number, the recurrence-equivalence exponent φ . On our prelude-recur-coda baseline we obtain $\varphi = 0.46$, so each shared recurrence is worth roughly half a unique block. The fitted φ responds to design choices, and our two case studies move it in opposite directions even though both lower validation loss. Hyperconnections raise φ to 0.65, while truncated BPTT lowers it to 0.38 by re-allocating compute toward more tokens and a wider model. Raw validation loss does not separate these two cases, but φ does. φ additionally reflects inference cost via the compute-optimal allocation, as $\Delta\varphi > 0$ narrows the compute-optimum and lowers per-token inference FLOPs, while $\Delta\varphi < 0$ widens it. We therefore propose $\Delta\varphi$ as a metric alongside validation loss for any new looped LM recipe or architecture, so that improvements are credited to the correct channel and recipes that quietly raise deployment cost are flagged.

References

- [1] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=HyzdRiR9Y7>.
- [2] Angeliki Giannou, Shashank Rajput, Jy-Yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped transformers as programmable computers. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 11398–11442. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/giannou23a.html>.
- [3] Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with latent thoughts: On the power of looped transformers. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=din0LGfZFd>.
- [4] Jonas Geiping, Sean Michael McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=S3GhJooWIC>.
- [5] Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, Qiyang Min, Hongzhi Huang, Xun Zhou, Wei Ye, Jiaheng Liu, Jian Yang, Yunfeng Shi, Chenghua Lin, Enduo Zhao, Tianle Cai, Ge Zhang, Wenhao Huang, Yoshua Bengio, and Jason Eshraghian. Scaling Latent Reasoning via Looped Language Models, November 2025.
- [6] Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyouon Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=QuqsEIVWIG>.
- [7] Tianyu Fu, Yichen You, Zekai Chen, Guohao Dai, Huazhong Yang, and Yu Wang. Think-at-hard: Selective latent iterations to improve reasoning language models, 2025. URL <https://arxiv.org/abs/2511.08577>.
- [8] Sean McLeish, Ang Li, John Kirchenbauer, Dayal Singh Kalra, Brian R. Bartoldson, Bhavya Kailkhura, Avi Schwarzschild, Jonas Geiping, Tom Goldstein, and Micah Goldblum. Teaching pretrained language models to think deeper with retrofitted recurrence, 2025. URL <https://arxiv.org/abs/2511.07384>.
- [9] Yeskendir Koishakenov, Aldo Lipani, and Nicola Cancedda. Encode, think, decode: Scaling test-time reasoning with recursive latent thoughts, 2025. URL <https://arxiv.org/abs/2510.07358>.
- [10] Hayden Prairie, Zachary Novack, Taylor Berg-Kirkpatrick, and Daniel Y. Fu. Parcae: Scaling laws for stable looped language models, 2026. URL <https://arxiv.org/abs/2604.12946>.
- [11] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack W. Rae, and Laurent Sifre. Training compute-optimal large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS ’22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.
- [12] Defa Zhu, Hongzhi Huang, Zihao Huang, Yutao Zeng, Yunyao Mao, Banggu Wu, Qiyang Min, and Xun Zhou. Hyper-connections. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=9FqARW7dwB>.

- [13] Jonas Knupp, Jan Hendrik Metzen, Jeremias Bohn, Georg Groh, and Kristian Kersting. Depth-recurrent attention mixtures: Giving latent reasoning the attention it deserves, 2026. URL <https://arxiv.org/abs/2601.21582>.
- [14] Ahmadreza Jeddi, Marco Ciccone, and Babak Taati. Loopformer: Elastic-depth looped transformers for latent reasoning via shortcut modulation. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=RzYXb5YWBs>.
- [15] Abbas Zeitoun, Lucas Torroba-Hennigen, and Yoon Kim. Hyperloop transformers, 2026. URL <https://arxiv.org/abs/2604.21254>.
- [16] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. URL <https://arxiv.org/abs/2001.08361>.
- [17] Andrej Karpathy. nanochat: The best ChatGPT that \$100 can buy, 2025. URL <https://github.com/karpathy/nanochat>.
- [18] Biao Zhang and Rico Sennrich. *Root mean square layer normalization*. Curran Associates Inc., Red Hook, NY, USA, 2019.
- [19] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomput.*, 568(C), February 2024. ISSN 0925-2312. doi: 10.1016/j.neucom.2023.127063. URL <https://doi.org/10.1016/j.neucom.2023.127063>.
- [20] Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Peter Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, Rodolphe Jenatton, Lucas Beyer, Michael Tschannen, Anurag Arnab, Xiao Wang, Carlos Riquelme Ruiz, Matthias Minderer, Joan Puigcerver, Utku Evci, Manoj Kumar, Sjoerd Van Steenkiste, Gamaleldin Fathy Elsayed, Aravindh Mahendran, Fisher Yu, Avital Oliver, Fantine Huot, Jasmijn Bastings, Mark Collier, Alexey A. Gritsenko, Vighnesh Birodkar, Cristina Nader Vasconcelos, Yi Tay, Thomas Mensink, Alexander Kolesnikov, Filip Pavetic, Dustin Tran, Thomas Kipf, Mario Lucic, Xiaohua Zhai, Daniel Keysers, Jeremiah J. Harmsen, and Neil Houlsby. Scaling vision transformers to 22 billion parameters. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 7480–7512. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/dehghani23a.html>.
- [21] David R. So, Wojciech Mańke, Hanxiao Liu, Zihang Dai, Noam Shazeer, and Quoc V. Le. Primer: searching for efficient transformers for language modeling. In *Proceedings of the 35th International Conference on Neural Information Processing Systems*, NIPS ’21, Red Hook, NY, USA, 2021. Curran Associates Inc. ISBN 9781713845393.
- [22] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=mZn2Xyh9Ec>.
- [23] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: fast and accurate attention with asynchrony and low-precision. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS ’24, Red Hook, NY, USA, 2024. Curran Associates Inc. ISBN 9798331314385.
- [24] Kaiyue Wen, Xingyu Dang, Kaifeng Lyu, Tengyu Ma, and Percy Liang. Fantastic pretraining optimizers and where to find them 2.1: Hyperball optimization, 12 2025. URL <https://tinyurl.com/muonh>.
- [25] Keller Jordan, Yuchen Jin, Vlado Boza, You Jiacheng, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon/>.

- [26] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- [27] Liliang Ren, Yang Liu, Yelong Shen, and Weizhu Chen. Rethinking language model scaling under transferable hypersphere optimization, 2026. URL <https://arxiv.org/abs/2603.28743>.
- [28] Ge Yang, Edward Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tuning large neural networks via zero-shot hyperparameter transfer. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 17084–17097. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/8df7c2e3c3c3be098ef7b382bd2c37ba-Paper.pdf.
- [29] Anton Lozhkov, Loubna Ben Allal, Leandro von Werra, and Thomas Wolf. Fineweb-edu: the finest collection of educational content, 2024. URL <https://huggingface.co/datasets/HuggingFaceFW/fineweb-edu>.
- [30] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023. URL <https://arxiv.org/abs/2307.09288>.
- [31] Peter J. Huber. Robust Estimation of a Location Parameter. *The Annals of Mathematical Statistics*, 35(1):73 – 101, 1964. doi: 10.1214/aoms/1177703732. URL <https://doi.org/10.1214/aoms/1177703732>.
- [32] Jorge Nocedal. Updating quasi-newton matrices with limited storage. *Mathematics of Computation*, 35(151):773–782, 1980. ISSN 00255718, 10886842. URL <http://www.jstor.org/stable/2006193>.
- [33] Tamay Besiroglu, Ege Erdil, Matthew Barnett, and Josh You. Chinchilla scaling: A replication attempt, 2024. URL <https://arxiv.org/abs/2404.10102>.
- [34] Makoto Shing, Masanori Koyama, and Takuya Akiba. Diffusionblocks: Block-wise neural network training via diffusion interpretation, 2026. URL <https://arxiv.org/abs/2506.14202>.
- [35] Nikhil Sardana, Jacob Portes, Sasha Dobov, and Jonathan Frankle. Beyond chinchilla-optimal: accounting for inference in language model scaling laws. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- [36] Nicholas Roberts, Sungjun Cho, Zhiqi Gao, Tzu-Heng Huang, Albert Wu, Gabriel Orlanski, Avi Trost, Kelly Buchanan, Aws Albarghouthi, and Frederic Sala. Test-time scaling makes overtraining compute-optimal, 2026. URL <https://arxiv.org/abs/2604.01411>.
- [37] Gemma Team. Gemma 2: Improving open language models at a practical size, 2024. URL <https://arxiv.org/abs/2408.00118>.
- [38] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *2015 IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015. doi: 10.1109/ICCV.2015.123.

- [39] Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Yitzhak Gadre, Hritik Bansal, Etash Kumar Guha, Sedrick Keh, Kushal Arora, Saurabh Garg, Rui Xin, Niklas Muennighoff, Reinhard Heckel, Jean Mercat, Mayee F Chen, Suchin Gururangan, Mitchell Wortsman, Alon Albalak, Yonatan Bitton, Marianna Nezhurina, Amro Kamal Mohamed Abbas, Cheng-Yu Hsieh, Dhruba Ghosh, Joshua P Gardner, Maciej Kilian, Hanlin Zhang, Rulin Shao, Sarah M Pratt, Sunny Sanyal, Gabriel Ilharco, Giannis Daras, Kalyani Marathe, Aaron Gokaslan, Jieyu Zhang, Khyathi Chandu, Thao Nguyen, Igor Vasiljevic, Sham M. Kakade, Shuran Song, Sujay Sanghavi, Fartash Faghri, Sewoong Oh, Luke Zettlemoyer, Kyle Lo, Alaaeldin El-Nouby, Hadi Pouransari, Alexander T Toshev, Stephanie Wang, Dirk Groeneveld, Luca Soldaini, Pang Wei Koh, Jenia Jitsev, Thomas Kollar, Alex Dimakis, Yair Carmon, Achal Dave, Ludwig Schmidt, and Vaishaal Shankar. Datacomp-LM: In search of the next generation of training sets for language models. In *The Thirty-eighth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL <https://openreview.net/forum?id=CNWdWn47IE>.
- [40] Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In Regina Barzilay and Min-Yen Kan, editors, *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL <https://aclanthology.org/P17-1147/>.
- [41] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL <https://aclanthology.org/Q19-1026/>.
- [42] Jonathan Berant, Andrew Chou, Roy Frostig, and Percy Liang. Semantic parsing on Freebase from question-answer pairs. In David Yarowsky, Timothy Baldwin, Anna Korhonen, Karen Livescu, and Steven Bethard, editors, *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1533–1544, Seattle, Washington, USA, October 2013. Association for Computational Linguistics. URL <https://aclanthology.org/D13-1160/>.
- [43] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In Katrin Erk and Noah A. Smith, editors, *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1525–1534, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1144. URL <https://aclanthology.org/P16-1144/>.
- [44] Jonathan H. Clark, Eunsol Choi, Michael Collins, Dan Garrette, Tom Kwiatkowski, Vitaly Nikolaev, and Jennimaria Palomaki. TyDi QA: A benchmark for information-seeking question answering in typologically diverse languages. *Transactions of the Association for Computational Linguistics*, 8:454–470, 2020. doi: 10.1162/tacl_a_00317. URL <https://aclanthology.org/2020.tacl-1.30/>.
- [45] Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don’t know: Unanswerable questions for SQuAD. In Iryna Gurevych and Yusuke Miyao, editors, *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 784–789, Melbourne, Australia, July 2018. Association for Computational Linguistics. doi: 10.18653/v1/P18-2124. URL <https://aclanthology.org/P18-2124/>.
- [46] Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In Jill Burstein, Christy Doran, and Tamar Solorio, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2368–2378, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1246. URL <https://aclanthology.org/N19-1246/>.

- [47] Siva Reddy, Danqi Chen, and Christopher D. Manning. CoQA: A conversational question answering challenge. *Transactions of the Association for Computational Linguistics*, 7:249–266, 2019. doi: 10.1162/tacl_a_00266. URL <https://aclanthology.org/Q19-1016/>.
- [48] Arkil Patel, Satwik Bhattamishra, and Navin Goyal. Are NLP models really able to solve simple math word problems? In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou, editors, *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2080–2094, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.168. URL <https://aclanthology.org/2021.naacl-main.168/>.
- [49] Shen-yun Miao, Chao-Chun Liang, and Keh-Yih Su. A diverse corpus for evaluating and developing English math word problem solvers. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 975–984, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.92. URL <https://aclanthology.org/2020.acl-main.92/>.
- [50] Rik Koncel-Kedziorski, Subhro Roy, Aida Amini, Nate Kushman, and Hannaneh Hajishirzi. MAWPS: A math word problem repository. In Kevin Knight, Ani Nenkova, and Owen Rambow, editors, *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1152–1157, San Diego, California, June 2016. Association for Computational Linguistics. doi: 10.18653/v1/N16-1136. URL <https://aclanthology.org/N16-1136/>.
- [51] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads, 2022. URL <https://arxiv.org/abs/2209.11895>.
- [52] Nikunj Saunshi, Stefani Karp, Shankar Krishnan, Sobhan Miryoosefi, Sashank J. Reddi, and Sanjiv Kumar. On the inductive bias of stacking towards improving reasoning. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=3ZAfFoAcUI>.
- [53] Aarohi Srivastava, Abhinav Rastogi, et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL <https://openreview.net/forum?id=uyTL5Bvosj>.
- [54] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge, 2018. URL <https://arxiv.org/abs/1803.05457>.
- [55] David Heineman, Valentin Hofmann, Ian Magnusson, Yuling Gu, Noah A. Smith, Hannaneh Hajishirzi, Kyle Lo, and Jesse Dodge. Signal and noise: A framework for reducing uncertainty in language model evaluation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=sAFottNlra>.

A Extended Related Work

In this section, we expand the main-text Related Work (Section 2).

Scaling laws. Kaplan et al. [16] established power-law relations between loss, model size, and training tokens. Hoffmann et al. [11] refined the allocation (Chinchilla) and found that compute-optimal training scales parameters and tokens at roughly equal rates with compute. Subsequent work has examined learning-rate transfer [27, 28] and inference-aware scaling that trades training tokens for inference cost [35, 36]. We extend these analyses to looped architectures.

Iso-parameter scaling law. Concurrent work by Prairie et al. [10] fits an iso-parameter scaling law at fixed unique parameter count N , with depth, per-token inference FLOPs, and KV cache memory growing with recurrence count μ_{rec} . At the core of their framework is the effective-parameter accounting $N_{\text{eff}} = \mu_{\text{rec}} N$, where recurrences multiply the full unique parameter count, prelude and coda included. In contrast, we separate parameters into N_{once} and N_{rec} .

Additionally, three methodological details affect direct comparison. (1) Prairie et al. [10] sample per-step recurrence counts from a distribution with mean μ_{rec} , while we fix r architecturally. (2) They use truncated BPTT, which reduces training FLOPs. We train our main grid under full BPTT to keep training and inference FLOPs aligned with the matched non-looped baseline, and probe the truncated-BPTT alternative separately (Section 5.1 shows it lowers φ). (3) Their default input injection is diagonal ($\mathcal{O}(d)$ parameters). Ours is a linear map $W_{\text{inject}} \in \mathbb{R}^{d \times 2d}$ with a small FLOPs cost (see Section 3.2). Their diagonal-injection layer remains untested in our framework. Overall, the two scaling laws are complementary and answer different questions.

A scaling law that matches parameters, FLOPs, and memory simultaneously remains an open direction. Knupp et al. [13] achieve such matching, introducing FLOP-neutral capacity through sparse MoEs alongside depth attention and per-recurrence expert routing layered onto every block. Extending their method to a scaling law setup would be interesting future work.

Test-time compute scaling. Iterating a shared block at inference time is one of the main promises of looped transformers. More loops yields more compute per token without growing parameters. Prior compute-matched studies of looped LMs nonetheless use low recurrence counts ($r \leq 4$ in [5–7]), because each additional loop carries a large training-FLOPs cost while adding less capacity than unique parameters. Our $\varphi = 0.46$ quantifies this directly. Eight loops are worth ≈ 2.6 unique blocks at matched depth, far below the eight that the inference cost accounts for. The implication is that effectively scaling test-time compute through more loops requires raising φ , not just r .

A finer-grained variant is per-token adaptive compute. Not all tokens are equally hard to predict, so a looped model can in principle vary the recurrence count per token at inference. *Per-token early exit* realises this idea, looping on hard tokens and exiting early on easy ones [1, 4, 5, 7]. In practice this has not delivered wall-clock speedups yet. Parallel prefill and batched decoding assume all tokens run the same number of layers per step, and variable-depth routing breaks this uniformity (KV cache entries are also missing for some loops). Pre-determined routing schemes such as Mixture-of-Recursions [6] (per-token) and LoopFormer [14] (per-sequence) restore batching at the cost of either causality issues during routing or limited advantages since all tokens in a sequence share one budget. Until per-token adaptive compute delivers wall-clock gains at inference, it does not raise the worth of a recurrence beyond what φ already captures at training time.

Test-time loop extrapolation. The optimistic picture from synthetic algorithmic tasks [2], of training at low recurrence counts and extrapolating to greater depth at inference, has not transferred cleanly to general language modelling. Geiping et al. [4], Prairie et al. [10] train their models with sampled recurrence counts extending to large values, to enable test-time scaling. However, Prairie et al. [10] fit a joint training–inference scaling law whose test-time component is a saturating exponential $\mathcal{L}(T) = \mathcal{L}_{\infty} + Z \exp(-zT/\mu_{\text{rec}})$ that plateaus at $T \approx \mu_{\text{rec}}$. Thus, the mean recurrence in training caps the test-time frontier. Ouro [5] was trained with an anytime-prediction gate and reports no inference-time gains beyond the trained depth either. Taken together, effective inference depth in trained looped LMs concentrates near the training depth distribution rather than extrapolating freely past it. We therefore treat r as an architectural, not a test-time, scaling axis. Until depth extrapolation works, it likewise does not raise the worth of a recurrence beyond what φ captures at training time.

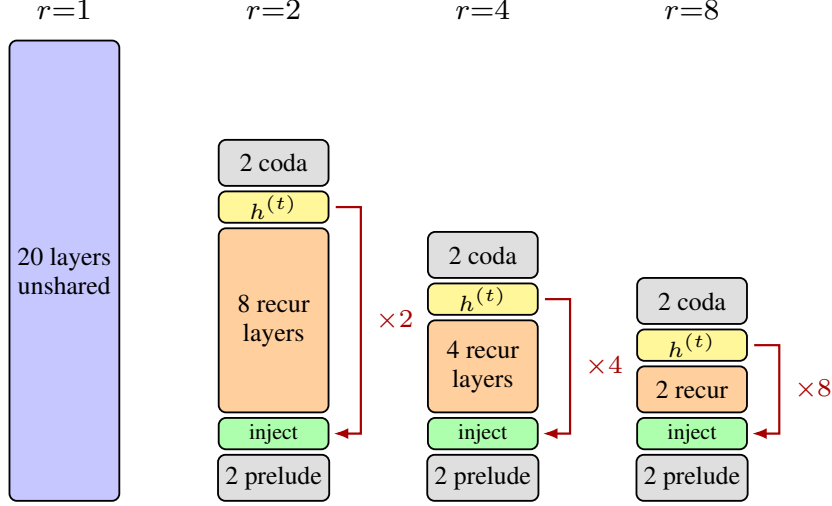


Figure 5: Architecture schematic for $r \in \{1, 2, 4, 8\}$ at shared effective depth 20. The recurrent block (orange) is applied r times per forward pass and writes its output back into the latent state $h^{(t)}$ (yellow) via the injection layer (green). Prelude and coda (grey) are unshared.

Table 4: Transformer architecture.

Component	Details
Sequence length	$T = 2048$ tokens
Attention	Full causal self-attention, no sliding window, no dropout
Attention backend	FlashAttention-2 [22] on A100, FlashAttention-3 [23] on H100
Position encoding	Rotary embeddings [19], base $\theta = 10,000$
Head dimension	$d_{\text{head}} = 128, n_{\text{head}} = d_{\text{model}}/128$
QK normalisation	Functional RMSNorm on q, k before attention [20]
MLP activation	Squared ReLU [21], hidden dim = $4d_{\text{model}}$
Normalisation	Learnable RMSNorm [18], pre-norm, $\epsilon=10^{-6}$
Biases	None (all linear layers bias-free)
Embeddings	Untied wte and lm_head, token embeddings cast to bf16
Vocabulary	Llama 2 tokenizer [30], padded $32008 \rightarrow 32064$ for tensor-core alignment
Logit softcap	$z = 15 \cdot \tanh(\text{logits}/15)$ [37], applied in fp32 before the loss
Dropout	None

B Compute Resources

Experiments ran on a mix of A100-80GB and H100-80GB GPUs. The 116-run iso-depth grid and the two case studies account for the bulk of the budget. The $s=34$ extrapolation pair at 47B tokens also added substantial cost. The full project, including exploratory configurations, failed runs, and side experiments on the same accounts, consumed approximately 5,000 GPU-hours.

C Model Architecture

C.1 Implementation Details

All architectures are decoder-only transformers built on top of nanochat [17], using the same pre-norm block summarised in Table 4. The layer partition for each r is $\ell_{\text{prelude}} + r \cdot \ell_{\text{recur}} + \ell_{\text{coda}} = 20$ with $(\ell_{\text{prelude}}, \ell_{\text{coda}}) = (2, 2)$ for $r > 1$, so $\ell_{\text{recur}} = 16/r$ evaluates to $\{8, 4, 2\}$ for $r \in \{2, 4, 8\}$. Figure 5 visualises the four stacks.

Table 5: Input-injection ablation at $s=10$, $r=4$, $C = 10^{18}$ FLOPs. Tokens differ across variants because the parameter-free and hyperconnect methods save the linear injection’s FLOPs overhead.

Injection variant	Tokens trained	Val. loss (nats)
Linear (default)	955M	2.793
Additive	973M	2.797
Passthrough	973M	7.400
Hyperconnections ($K=2$)	973M	2.757

Each transformer block computes

$$\hat{x} = x + \text{Attn}(\text{RMSNorm}(x)), \quad x_{\text{out}} = \hat{x} + \text{MLP}(\text{RMSNorm}(\hat{x})).$$

In addition to the pre-norms inside each block, three model-level RMSNorms are applied on the residual stream: one after the token embedding, one at the end of every recurrence iteration (so the state handed to the next iteration or to the coda has controlled scale), and one before the `lm_head`.

Input injection (looped only). For each looped architecture ($r > 1$), every recurrence iteration begins with a linear injection step

$$u^{(t)} = W_{\text{inject}} [e \parallel h^{(t)}], \quad W_{\text{inject}} \in \mathbb{R}^{d \times 2d}, \quad (5)$$

where e is the prelude output (constant across recurrences), $h^{(t)}$ is the recurrent state at iteration t with $h^{(0)} = e$, and W_{inject} is initialised as $[I \parallel 0]$ so that $u^{(0)} \approx e$ at the start of training. Appendix C.2 ablates this choice against additive-residual and no-injection variants.

Initialisation. Token embeddings are drawn from $\mathcal{N}(0, 1)$ and then cast to bf16. The LM head is $\mathcal{N}(0, 10^{-3})$. Attention and MLP weights use $\mathcal{U}(-a, a)$ with $a = \sqrt{3}/\sqrt{d_{\text{model}}}$ (equivalently, the same standard deviation $1/\sqrt{d_{\text{model}}}$ as the matched normal but with bounded tails), except `mlp.c_proj` which uses $a = \sqrt{3}/\sqrt{4d_{\text{model}}}$ (Kaiming fan-in [38] over its input width $4d_{\text{model}}$). The injection layer is initialised as $[I \parallel 0]$, and all RMSNorm scales are initialised to one.

C.2 Input-Injection Ablation

Our default injection is the linear map of Equation 5, following the concatenation-injection design of Geiping et al. [4]. The linear map adds $2d_{\text{model}}^2$ parameters and, applied once per recurrence, a non-negligible FLOPs overhead that is paid back only if it improves quality. We verify this at the reference configuration ($s=10$, $r=4$, target compute 10^{18} FLOPs) against two parameter-free alternatives:

- **Passthrough** ($u^{(t)} = h^{(t)}$): no injection, recurrence is depth-only with $h^{(0)}$ initialised from the prelude output.
- **Additive** ($u^{(t)} = h^{(t)} + e$): parameter-free residual injection with $h^{(0)} = 0$, so the first iteration sees $u^{(0)} = e$.

All variants use the same target FLOPs budget, so the parameter-free alternatives train on more tokens than the linear injection (973M vs. 955M, a $\sim 2\%$ data advantage from the saved injection FLOPs). Results are summarised in Table 5. Passthrough fails to train, showing that some form of injection is essential at this scale. Additive is competitive but trails the linear injection by 0.004 nats despite its token advantage. We therefore adopt the linear injection for all reported scaling-law runs. Its FLOPs overhead is accounted for in n_i in Equation 1 and thus included in every iso-FLOPs comparison.

Hyperconnections [12] are only listed for reference. We did not adopt them in the main scaling-law grid. They are used in Section 5.2 to test whether they improve φ .

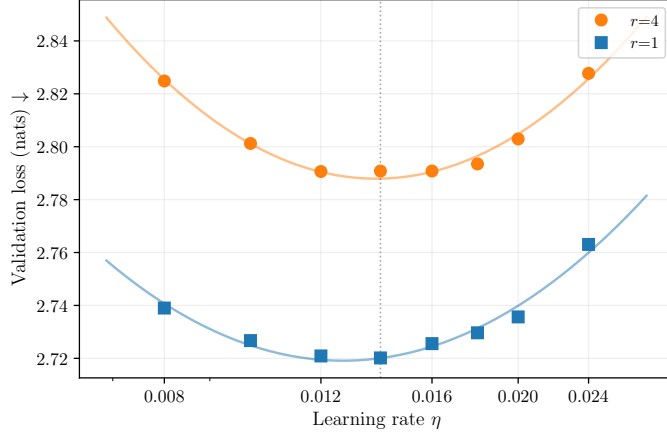


Figure 6: Learning rate sweep at $s=10$ (ratio 10, $B = 256K$). Both architectures exhibit a clear U-shaped loss landscape with a shared optimum near $\eta^* \approx 0.014$. The dotted vertical line marks $\eta = 0.014$, the base LR adopted for all scaling-law runs.

D Hyperparameter Tuning

D.1 Learning Rate Sweep

We sweep the MuonH [24, 27] matrix learning rate at $s=10$ with a tokens-per-parameter ratio of 10 ($\sim 1B$ training tokens) and batch size $B = 262,144$ (256K), independently for each architecture, using eight LR values per architecture in the range $\eta \in [0.008, 0.024]$ with extra density around the optimum. The batch size was chosen from a separate sweep at the same reference configuration over $B \in \{256K, 512K, 1M\}$ tokens, where 256K yielded uniformly lower loss for both architectures.

Both architectures converge to a shared optimum near $\eta^* \approx 0.014$, which we adopt as the base learning rate for all subsequent experiments (Figure 6).

D.2 Transfer Validation

Under the HyperP framework [27] the *base* learning rate η_{base} (the value fed into HyperP before its width and data corrections are applied) should be invariant to both width and training horizon (after the $D^{-0.32}$ data-scaling correction of the HyperP LR rule, with D the training-token count). We verify both claims by repeating the LR sweep under varied conditions and measuring the regret: the loss penalty of using $\eta_{\text{base}} = 0.014$ instead of the per-condition optimum.

Width transfer. We sweep at $s \in \{8, 10, 14\}$ (ratio 10, $B = 256K$). Figure 7 (left column) shows the regret U-curves in base LR space: all minima cluster near $\eta_{\text{base}} = 0.014$ with a maximum regret of 0.004 nats ($s=8$ looped). As a lightweight sanity check past the sweep range, we run the two candidate LRs $\eta_{\text{base}} \in \{0.012, 0.014\}$ at $s=18$ for the looped architecture and find 0.014 marginally better (2.473 vs. 2.476 nats), confirming that 0.014 remains near-optimal.

Data scaling. We sweep at $s=10$ ($B = 256K$) with ratios $\{10, 20, 40\}$, spanning a $4\times$ range in training tokens. If the $T^{-0.32}$ exponent is correct, the data-scaling correction adjusts the effective LR automatically and the optimal base LR should remain constant. Figure 7 (right column) confirms this: regret at $\eta_{\text{base}} = 0.014$ stays below 0.005 nats across all ratios for both architectures.

E Iso-Depth Grid

For each compute budget we sweep model width to find the compute-optimal point at every recurrence count $r \in \{1, 2, 4, 8\}$. Table 6 reports the unique non-embedding parameter count $N(s, r)$ at each width and the training tokens D used at each (budget, width) cell. Looped models train on slightly fewer tokens due to input injection compute overhead. Empty cells are untested widths.

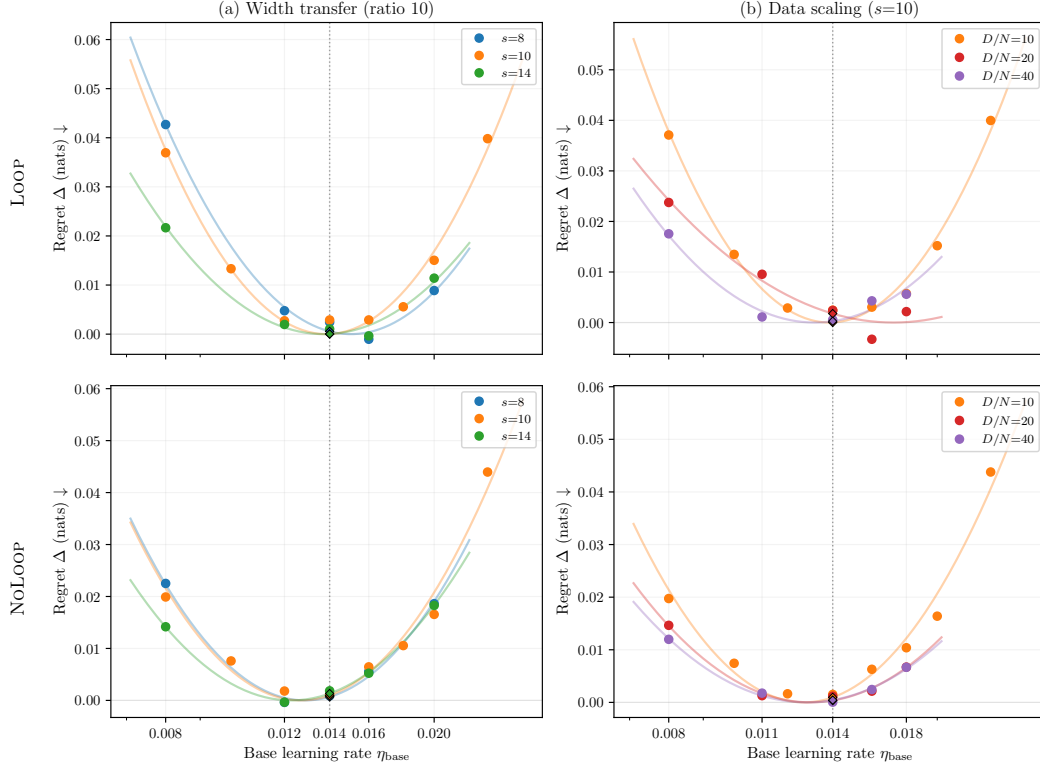


Figure 7: Transfer validation. Regret (loss above the per-condition optimum) vs. base learning rate η_{base} . Vertical dotted line marks $\eta_{\text{base}} = 0.014$; diamond markers show the regret at that point. Rows split by architecture (looped $r=4$, top; non-looped $r=1$, bottom). All conditions incur ≤ 0.005 nats regret, so $\eta_{\text{base}} = 0.014$ transfers cleanly across width and training horizon.

Table 6: Iso-compute grid. Left: unique non-embedding parameter count N (M) per (width, recurrence) cell. Right: training tokens (B) per (width, budget) cell for $r = 1$.

s	Unique params N (M)				Training tokens (B) per budget (FLOPs)					
	$r=1$	$r=2$	$r=4$	$r=8$	$4.64 \cdot 10^{17}$	10^{18}	$2.15 \cdot 10^{18}$	$4.64 \cdot 10^{18}$	10^{19}	$2.15 \cdot 10^{19}$
6	35.4	21.5	14.5	10.9	0.98	2.10	—	—	—	—
8	62.9	38.3	25.7	19.4	0.64	1.36	2.95	—	—	—
10	98.3	59.8	40.2	30.3	0.45	0.97	2.08	4.49	—	—
12	141.6	86.1	57.8	43.7	0.34	0.72	1.55	3.34	7.13	—
14	192.7	117.2	78.7	59.4	—	0.56	1.20	2.59	—	—
16	251.7	153.1	102.8	77.6	—	—	0.96	2.07	4.43	—
18	318.6	193.8	130.1	98.2	—	—	—	1.70	3.62	7.78
20	393.3	239.2	160.6	121.3	—	—	—	1.42	3.05	—
24	566.3	344.5	231.2	174.6	—	—	—	—	2.20	4.71
28	770.8	468.9	314.7	237.7	—	—	—	—	—	3.58
34	1136.5	691.4	464.1	350.4	—	—	—	—	—	2.52

F Scaling Law Fit Diagnostics

We conduct robustness checks for the per-architecture Chinchilla fits (Equation 2) and the joint $(N_{\text{once}}, N_{\text{rec}}, D, r)$ law (Equation 3). Residuals for the per-architecture fits, aggregate residual statistics for the joint fit, the block-bootstrap procedure behind the φ confidence interval, and stability of φ across budget halves.

Optimisation details. The Huber-on-log objective of Equation 4 is non-convex, so for both the per-architecture and joint fits we take the best of 500 random L-BFGS-B restarts. Parameters are

Table 7: Joint-law residual statistics broken down by architecture (actual – predicted, in nats). Each architecture contributes 29 runs to the joint fit.

Architecture	n	mean resid	max resid	RMSE
$r=1$	29	−0.001	0.018	0.009
$r=2$	29	+0.004	0.036	0.011
$r=4$	29	−0.006	0.023	0.011
$r=8$	29	+0.001	0.029	0.010

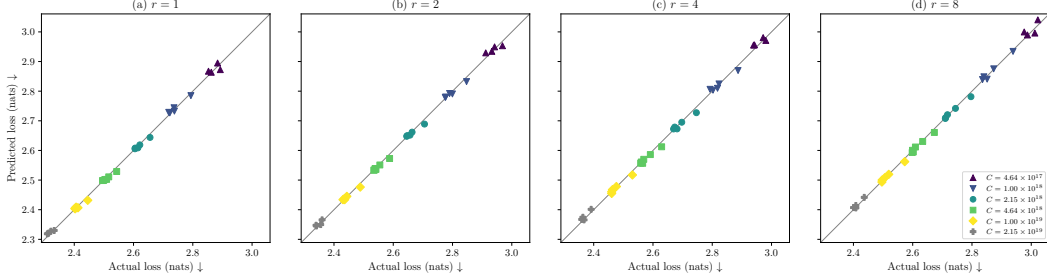


Figure 8: Per-architecture Chinchilla fit quality: predicted vs. actual validation loss, one panel per architecture ($r \in \{1, 2, 4, 8\}$). Markers redundantly encode the compute budget by both shape and colour. Points cluster tightly around the identity line across all four architectures.

constrained to the box $a, b \in [-5, 35]$, $\alpha, \beta \in [0, 2.5]$, $e \in [-3, 2]$ (and $\varphi \in [-3, 3]$ for the joint fit), with starting points drawn uniformly inside the box and a per-restart cap of 10,000 iterations.

F.1 Per-Architecture and Joint Fit Residuals

Figure 8 shows predicted vs. actual validation loss for the four per-architecture Chinchilla fits. Points cluster tightly around the diagonal with a maximum residual below 0.007 nats. Figure 9 plots the same residuals against N and D and shows no systematic bias across either axis.

On the joint law, which fits all 116 runs with six shared parameters ($A, \alpha, B, \beta, E, \varphi$) rather than 20 per-architecture parameters, residuals are naturally larger but still small: $\max |\text{resid}| = 0.036$ nats, pooled $\text{RMSE} = 0.010$ nats, and $R^2 = 0.997$. Table 7 breaks the joint-fit residuals down by architecture. RMSE is comparable across r (0.009–0.011 nats) and per- r means lie within ± 0.006 nats of zero, so a single shared φ fits all four architectures with comparable accuracy and no architecture absorbs a disproportionate share of the residual mass.

F.2 Bootstrap Procedure

The 95% CI reported alongside the joint-law point estimate ($\varphi = 0.46$, $[0.41, 0.53]$) is a block bootstrap over *(budget, architecture) cells*: each cell groups all widths trained at a given (compute budget, r) pair, so resampling respects the experimental block structure rather than treating individual runs as independent. We draw 200 resamples with replacement of the non-empty cells (6 budgets $\times$ 4 architectures), refit the joint law on each resample, and report the 2.5th / 97.5th percentiles of the resulting φ distribution. Zero resamples reach either $\varphi = 0$ or $\varphi = 1$. We do not bootstrap restricted variants of the law.

F.3 Stability Across Budget Halves

Refitting the joint law separately on the low-budget half ($C \leq 2.15 \times 10^{18}$, $n=56$ runs) gives $\varphi = 0.44$, and refitting on the high-budget half ($C \geq 4.64 \times 10^{18}$, $n=60$ runs) gives $\varphi = 0.49$. φ therefore does not drift with scale inside our compute window, and the bootstrap CI comfortably contains both half-window estimates.

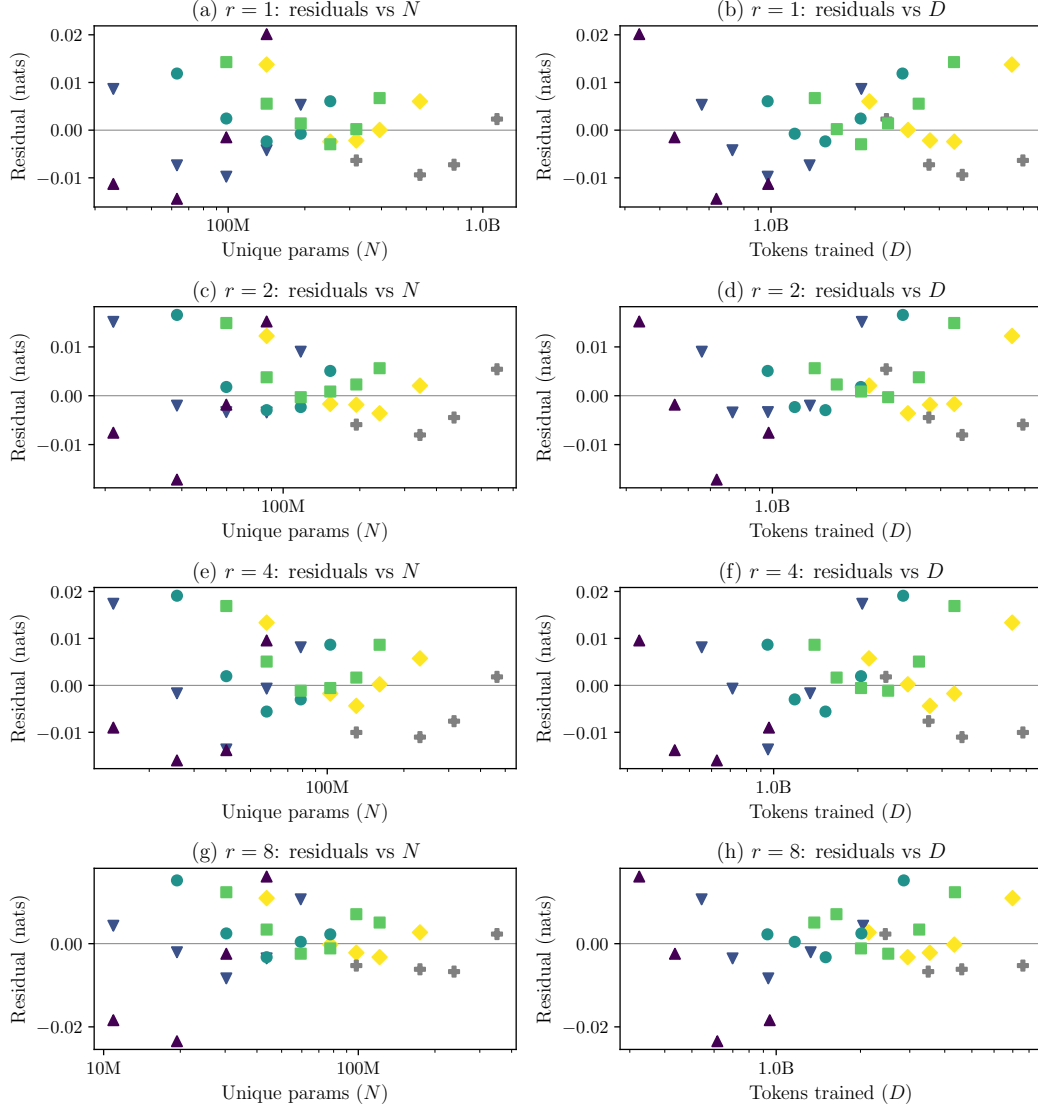


Figure 9: Per-architecture Chinchilla fit residuals (actual – predicted) vs. unique parameters N (left column) and tokens D (right column). Rows are the four architectures $r \in \{1, 2, 4, 8\}$. Markers encode the compute budget by both shape and colour.

F.4 Example: Equivalent Model Sizes at $r=4$

We compare a looped $r=4$ model and a non-looped baseline at the same width and effective depth. The resulting unique-parameter and effective-parameter counts are purely architectural ratios.

Take any $r=1$ configuration with $N(r=1) = 1\text{B}$ and width d_{model} . At the same d_{model} and $\ell_{\text{eff}}=20$ with $(\ell_{\text{prelude}}, \ell_{\text{coda}}) = (2, 2)$, the $r=4$ variant uses $\ell_{\text{recur}}=4$ and the unique-parameter ratio (from the $N_{\text{once}}, N_{\text{rec}}$ split in Section 3.2, dropping the small injection term $n_i = n_b/6$) is

$$\frac{N(r=4)}{N(r=1)} = \frac{(\ell_{\text{prelude}} + \ell_{\text{recur}} + \ell_{\text{coda}}) n_b}{\ell_{\text{eff}} n_b} = \frac{8}{20} = 0.40,$$

so the $r=4$ variant has $0.40 \times$ the unique parameters of the non-looped model: $\approx 410\text{M}$ including the injection term. The effective-parameter ratio under the joint law (Equation 3) is

$$\frac{N_{\text{eff}}(r=4; \varphi)}{N(r=1)} = \frac{N_{\text{once}} + 4^\varphi N_{\text{rec}}}{\ell_{\text{eff}} n_b} \approx \frac{4 + 4^\varphi \cdot 4}{20} \stackrel{\varphi=0.46}{\approx} 0.58,$$

giving the $\approx 580\text{M}$ figure. Per-token training FLOPs depend only on the executed-layer count, which is identical at fixed d_{model} up to the $\sim 3\%$ injection overhead of Equation 1, so the $r=4$ model trains at the same per-step cost as the 1B non-looped baseline. The same calculation at any other reference size gives the same percentages.

G Case Study Details

G.1 Methods

Truncated backpropagation. We follow Prairie et al. [10] and set the gradient window to $r_{\text{bwd}} = \lceil r/2 \rceil$ for $r \in \{2, 4, 8\}$, giving $r_{\text{bwd}} \in \{1, 2, 4\}$. The forward pass is unchanged. After the i -th recurrence, the recurrent state $s^{(i)}$ is detached for all $i < r - r_{\text{bwd}}$, so gradients flow only through the last r_{bwd} iterations. Detached iterations skip the backward pass and save roughly half the FLOPs of a full recurrent iteration, which translates to about 30% fewer training FLOPs per token under our prelude-recur-coda partition,

$$F_{\text{train}}^{\text{trunc}}(r) = (2(r - r_{\text{bwd}}) + 6r_{\text{bwd}})(\ell_{\text{recur}}n_b + n_i) + 6(\ell_{\text{prelude}} + \ell_{\text{coda}})n_b. \quad (6)$$

The freed compute is reinvested as more tokens at fixed budget. The empirical median ratio across all (budget, r , s) cells is $D_{\text{trunc}}/D_{\text{full}} = 1.315$.

Hyperconnections. We use our own implementation of a looped transformer with hyperconnections [12]. Note that Zeitoun et al. [15] concurrently proposed a similar architecture. We replace the linear input-injection layer with $K=2$ parallel lane states $\ell^{(i)} \in \mathbb{R}^{K \times d_{\text{model}}}$ mixed at every recurrence iteration. Lanes are initialised by broadcasting the prelude output e across all K slots, $\ell^{(0)} = (e, \dots, e)$. At iteration $i \in \{0, \dots, r-1\}$ the recurrent block sees

$$u^{(i)} = \alpha_i \cdot \ell^{(i)}, \quad s^{(i)} = \text{RecurBlock}(u^{(i)}), \quad \ell^{(i+1)} = M_i \ell^{(i)} + \beta_i \otimes s^{(i)}, \quad (7)$$

with per-iteration mixing parameters $\alpha_i \in \mathbb{R}^K$, $M_i \in \mathbb{R}^{K \times K}$, $\beta_i \in \mathbb{R}^K$, for a total of $r(K^2 + 2K)$ scalars across all iterations (32 at $K=2$, $r=4$). The coda receives the sum-pooled lanes $\sum_k \ell_k^{(r)}$. We adopt the cyclic initialisation of Zhu et al. [12], $\alpha_i = \mathbf{e}_{i \bmod K}$, $M_i = I$, $\beta_i = \mathbf{1}$, so that the first training iteration reduces to a plain looped forward pass. All hyperconnect runs use full BPTT. The per-token training-FLOPs cost of the mixing operations is $6r(K^2 + 2K)d_{\text{model}}$ (forward $2\times$ plus backward $4\times$), accounted for in our budget calculations and negligible: two orders of magnitude smaller than the linear injection it replaces.

G.2 Fit Quality

Figure 10 shows predicted vs. actual validation loss for both case-study joint fits, the analogue of Figure 8 on the main grid. Points cluster on the diagonal in both panels, with overall RMSEs an order of magnitude below the inter-architecture spread.

The truncated-BPTT panel shows where the $R^2 = 0.983$ is lost. Most of the residual mass sits at $r=2$ ($r_{\text{bwd}} = 1$), where the joint law systematically under-predicts loss. Only a single recurrence receives a direct gradient, which likely undertrains the looping mechanism for that architecture. The $r \in \{4, 8\}$ rows are well-behaved. Refitting on $r \in \{4, 8\}$ alone (Table 3) raises R^2 from 0.983 to 0.996, removing the $r=2$ residual mass without changing φ .

The hyperconnections panel does not show a comparable architecture-specific structure. Residuals are uniformly small across r , and the fewer runs (83 vs. 116 in the main fit) and narrower compute span (four of six budgets) are the main sources of R^2 loss relative to the main joint fit.

G.3 Compute-Optimal Allocation Under the Case Studies

The shift in φ has predictable consequences for compute-optimal allocation, visible directly in the iso-FLOPs panels of Figure 4. Treating r as fixed and writing the joint law as $L = E + (A \cdot g_r^{-\alpha})N^{-\alpha} + BD^{-\beta}$ with $g_r = N_{\text{once}}/N + r^\varphi N_{\text{rec}}/N$, a higher φ raises g_r at $r > 1$, which lowers the effective parameter amplitude $A_{\text{eff}} = Ag_r^{-\alpha}$, which in turn lowers the compute-optimal width $N^*(C)$ for that r .

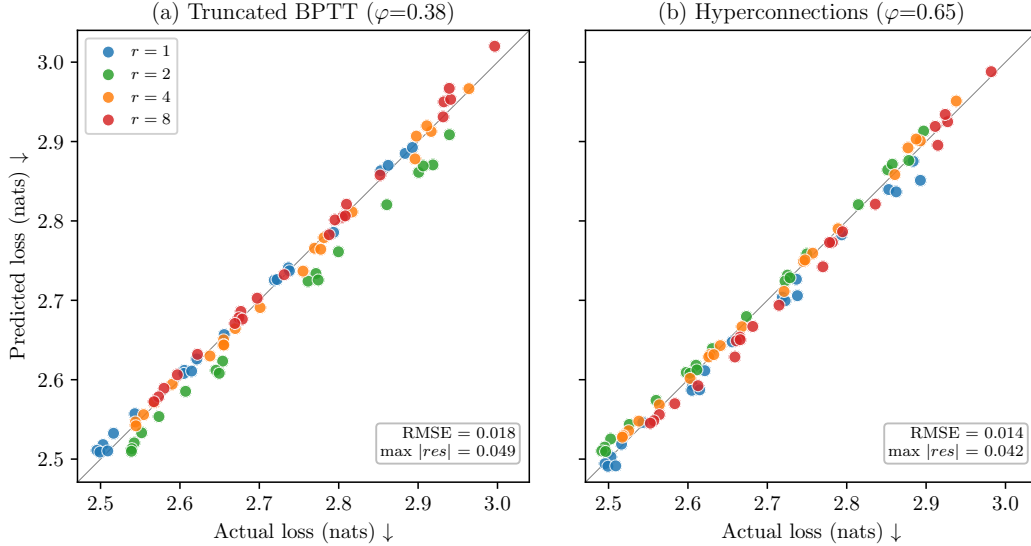


Figure 10: Case-study fit quality: predicted vs. actual validation loss under the joint-law refits of Table 3. Markers encode recurrence $r \in \{1, 2, 4, 8\}$ by colour. Annotation reports root-mean-square and maximum absolute residual.

Truncated BPTT. φ falls from 0.45 to 0.38, so g_r shrinks at every $r > 1$ and the compute-optimal width s^* widens further than under full BPTT. The picture is therefore one of larger looped models, consistent with the rightward shift of the truncated BPTT compute-optimal stars in Figure 4 (left). Wider compute-optimal models also raise per-token inference FLOPs, so the recipe trades training FLOPs for inference FLOPs at fixed deployment budget.

Hyperconnections. φ rises from 0.45 to 0.65, so g_r at $r > 1$ grows and the compute-optimal width s^* contracts. The compute-optimal stars on the hc panel of Figure 4 (right) sit at smaller N^* than the corresponding full-BPTT linear-injection stars at matched budgets. Lower N^* at the same compute also means lower per-token inference FLOPs, the converse of the truncated BPTT result.

The combined reading is that $\Delta\varphi$ alone determines the direction of compute-optimal width shifts. We observe both directions cleanly within the same architecture family, which validates the joint law as a budget-allocation tool, not just a goodness-of-fit summary.

H Downstream Evaluation Suite

H.1 Setup

Our downstream suite partitions tasks into five mechanistically motivated axes, each isolating a single capability dimension so that architectural biases can be read off directly. Tasks are sourced from the CORE benchmark [39], the Saunshi suite [3], and a small set of in-house probes. Per-task settings are in Table 8.

Few-shot counts were chosen to match or approximate the source benchmarks’ canonical settings. CoQA is reduced to 1-shot because the full-passage prompts frequently exceed the 2,048-token context used throughout pretraining. All four architectures share the same shot count and prompts on every task.

Axes and rationale.

- **Parametric knowledge.** Closed-book QA that requires recall of facts stored in weights, with no supporting passage: TriviaQA [40], NaturalQuestions [41], WebQuestions [42].

Table 8: Per-task settings. Type: MC = multiple choice, LM = language modelling (continuation log-likelihood). Continuation loss is reported throughout. Samples is the number of examples actually scored: all tasks are capped at 10,000 examples (TriviaQA and BigBench QA-Wikidata, with 17,944 and 20,321 examples in the source datasets, are uniformly subsampled).

Axis	Task	Samples	Shots	Type
Parametric knowledge	TriviaQA [40]	10,000	5	LM
	NaturalQuestions [41]	3,610	5	LM
	WebQuestions [42]	2,032	5	LM
Reading comp.	Lambda-OpenAI [43]	5,153	0	LM
	TydiQA-GoldP [44]	440	3	LM
	SQuADv2 [45]	5,928	3	LM
	DROP [46]	9,535	3	LM
	CoQA [47]	7,983	1	LM
Math word problems	SVAMP [48]	300	5	LM
	ASDiv [49]	2,305	5	LM
	MAWPS [50]	1,772	5	LM
Reasoning primitives	Induction head (in-house)	1,000	0	LM
	VarAssign d0 (math) [52]	1,000	5	LM
	VarAssign d0 (code) [52]	1,000	5	LM
	VarAssign d1 (math) [52]	1,000	5	LM
	VarAssign d1 (code) [52]	1,000	5	LM
Compositional symbolic	BigBench Dyck [53]	1,000	10	LM
	BigBench QA-Wikidata [53]	10,000	10	LM
	ARC-Easy [54]	2,376	10	MC
	BigBench CS-algorithms [53]	1,320	10	LM

- **Reading comprehension.** Extract or continue answer spans from an in-context passage: Lambda-OpenAI [43], TydiQA-GoldP [44], SQuADv2 [45], DROP [46], CoQA [47]. This probes in-context binding and multi-sentence extraction.
- **Math word problems.** Grade-school arithmetic in natural language: SVAMP [48], AS-Div [49], MAWPS [50]. This probes multi-step numeric chaining.
- **Reasoning primitives.** Minimal in-context symbolic operations. An induction-head probe following Olsson et al. [51] and four variable-assignment probes reimplemented from Saunshi et al. [52] (depth 0 and depth 1, each in math and code surface formats). *Variable assignment:* each example presents 5 direct integer assignments (depth 0) or 5 direct assignments plus 5 one-hop aliases with a 1-to-1 base-alias mapping (depth 1), in either a math format (“ $n=22$ ”) or a Python format (“ $n = 22$ ”), with English scaffolding. Values are drawn from [1, 25], and the answer is the queried variable’s integer value.
- **Compositional symbolic.** Multi-step structured manipulation over in-context sequences: BigBench Dyck-languages [53], BigBench QA-Wikidata [53], ARC-Easy [54], BigBench CS-algorithms [53].

H.2 Compute-Optimal Per-Axis Results

The scaling-law analysis summarises the sharing cost on validation loss, but not where that cost falls across downstream capabilities. We therefore re-evaluate every iso-FLOPs checkpoint at each $r \in \{1, 2, 4, 8\}$ on the five-axis downstream suite. Following Heineman et al. [55], we report per-token continuation loss on the continuation as the primary signal (Appendix H.5). We focus on the *compute-optimal* models, so at each FLOPs budget we pick the model with the lowest validation loss.

The results in Figure 11 split the five axes into three regimes.

Parametric knowledge tracks the validation-loss ordering. Parametric knowledge is closed-book recall and therefore capacity-bound. The $r=1$ baseline leads at every compute budget, and the gap grows monotonically with r , reaching 0.28 nats at $r=8$. This ordering matches the prediction from

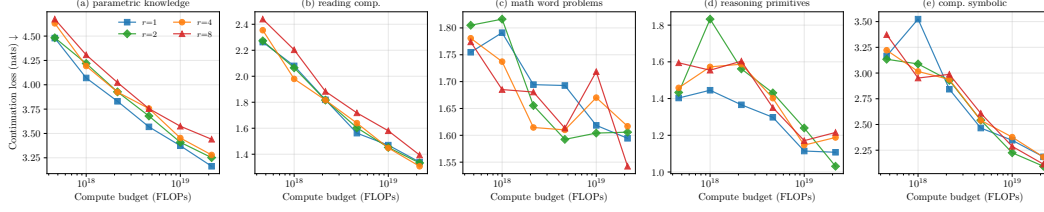


Figure 11: Compute-optimal downstream evaluation. Per-axis continuation loss at the r -specific checkpoint with lowest validation loss, per compute budget, for $r \in \{1, 2, 4, 8\}$. The five axes are defined in Appendix H. Lower is better.

$\varphi = 0.46$: more recurrences share more parameters, leaving less unique-parameter capacity for knowledge storage.

Reading comprehension and compositional symbolic close the gap. Reading comprehension and compositional symbolic close the gap between architectures seen on parametric knowledge. On reading comprehension, $r \in \{2, 4\}$ match $r=1$ and only $r=8$ trails (0.05–0.18 nats). On compositional symbolic, aggregates are roughly tied across r at all budgets, with mixed per-task outcomes (Appendix H.3). Looped variants lead on BigBench Dyck, $r=1$ leads on QA-Wikidata and ARC-Easy, and CS-algorithms is essentially tied.

Reasoning primitives and math word problems are unresolvable at our scale. Reasoning primitives and math word problems are the axes on which depth-recurrent models are predicted to win most strongly [3], yet neither resolves a per- r signal at our budgets. On reasoning primitives the $r=1$ baseline leads at nearly every budget. On math word problems, continuation loss improves with overall model quality but per- r separation falls inside noise. Both axes improve with validation loss in aggregate, but per- r separation is below our resolution, so these axes cannot drive architectural decisions at our scale. Reasoning tasks are too challenging for small models to show signal.

H.3 Per-Task Continuation Loss

The five-axis aggregates in the main text average over multiple tasks. Table 9 reports the underlying per-task continuation loss at the per-architecture compute-optimal checkpoint at the largest training budget $C = 2.15 \times 10^{19}$ FLOPs. The last column shows the dynamic range of $r=1$ continuation loss across the six budgets at the $r=1$ compute-optimal checkpoint of each budget, giving a sense of how much room each task improves with compute.

A few per-task patterns are consistent with the axis-level aggregates. On parametric knowledge, $r=1$ has the lowest loss on all three tasks with a monotone ordering across r , reproducing the validation-loss ordering. The reading-comprehension ordering varies task by task: TydiQA-GoldP, SQuADv2, DROP, and CoQA all favour $r=4$, while Lambada-OpenAI is monotone in $r=1$, consistent with the roughly flat reading-comp aggregate in Section H.2. On compositional symbolic, the looped variants lead on BigBench Dyck, while on QA-Wikidata and ARC-Easy $r=1$ leads, and BigBench CS-algorithms is essentially tied across r . On reasoning primitives, induction-head and var-assign d0 (math/code) carry most of the signal, with the d1 variants near random-guessing: $r=2$ leads on induction-head and on var-assign d0 (code) and d1 (code), $r=4$ on var-assign d0 (math), and $r=1$ on d1 (math). Math word problems are compressed within ~ 0.1 nats across r , so the small per-task differences should not be over-interpreted.

H.4 Per-Axis Continuation Loss versus Validation Loss

Figure 12 complements the compute-optimal summary of Section H.2 by plotting per-axis continuation loss against validation loss for every iso-Depth checkpoint. Each panel shows how one downstream axis tracks LM quality across the four architectures. Per- r curves show a different ordering compared to the main figure because the architectures reach a given val loss with different (N, D) allocations.

Table 9: Per-task continuation loss (nats, lower is better) at the per-architecture compute-optimal checkpoint at $C = 2.15 \times 10^{19}$ FLOPs. The last column shows the range of continuation loss across the six compute budgets at the $r=1$ compute-optimal checkpoint of each budget.

Axis	Task	$r=1$	$r=2$	$r=4$	$r=8$	$r=1$ range over C
Parametric knowledge	TriviaQA	3.251	3.346	3.381	3.497	3.251–4.732
	NaturalQuestions	3.158	3.226	3.257	3.393	3.158–4.237
	WebQuestions	3.079	3.183	3.199	3.431	3.079–4.469
Reading comp.	Lambda-OpenAI	1.910	1.934	1.991	2.116	1.910–3.243
	TydiQA-GoldP	0.631	0.624	0.564	0.605	0.631–1.299
	SQuADv2	1.001	0.990	0.924	1.020	1.001–2.060
	DROP	1.674	1.667	1.630	1.736	1.674–2.096
	CoQA	1.482	1.449	1.427	1.492	1.482–2.616
Math word problems	SVAMP	1.645	1.634	1.643	1.540	1.631–1.778
	ASDiv	1.635	1.657	1.675	1.634	1.635–1.892
	MAWPS	1.503	1.526	1.532	1.453	1.503–1.703
Reasoning primitives	Induction head	1.760	1.666	2.208	2.187	1.760–2.339
	VarAssign d0 (math)	0.790	0.816	0.715	0.812	0.731–1.237
	VarAssign d0 (code)	0.891	0.612	0.787	0.815	0.823–1.162
	VarAssign d1 (math)	0.986	1.127	1.090	1.123	0.902–1.280
	VarAssign d1 (code)	1.110	0.935	1.142	1.141	0.990–1.232
Compositional symbolic	BigBench Dyck	3.902	3.419	3.733	3.264	3.902–6.943
	BigBench QA-Wikidata	1.737	1.808	1.874	1.999	1.737–3.544
	ARC-Easy	1.977	2.020	2.010	2.088	1.977–3.093
	BigBench CS-algorithms	1.127	1.127	1.117	1.105	1.123–1.286

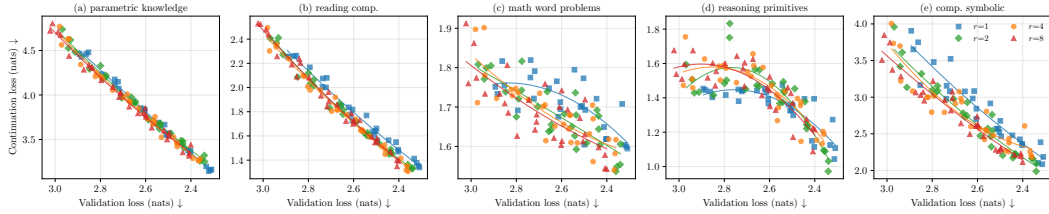


Figure 12: Per-axis continuation loss vs. validation loss for all iso-FLOPs checkpoints, coloured by recurrence count $r \in \{1, 2, 4, 8\}$. Curves are per- r quadratic fits, and the x-axis is inverted (lower-loss models on the right).

H.5 Accuracy versus Continuation Loss

At small scales many tasks are near the random-chance accuracy floor, where accuracy is a coarse, bimodal signal. Following Heineman et al. [55] we report continuation loss throughout. Figure 13 shows the correlation of each metric with validation loss across all iso-FLOPs checkpoints: continuation loss tracks validation loss nearly linearly, while the accuracy aggregate is noisier and flat for small scales.

I Extrapolation Beyond the Grid

To test whether the iso-depth findings hold past our grid, we train an $r=1$ and an $r=4$ run at $s=34$ (width $d_{\text{model}} = 2,176$) on 47B tokens, $\sim 20\times$ the top of our grid in training compute. All training hyperparameters match the grid runs (Section 4.1) except for the batch size, which we raise to $B = 524,288$ tokens to reduce gradient variance at this scale. This pair is trained at matched tokens rather than matched FLOPs, giving the looped model a $\sim 3\%$ training-FLOPs advantage from its injection-layer overhead (Equation 1). The gaps reported below are therefore conservative estimates of the iso-FLOPs gap. The looped run still completes in less wall-clock time thanks to its smaller unique parameter count.

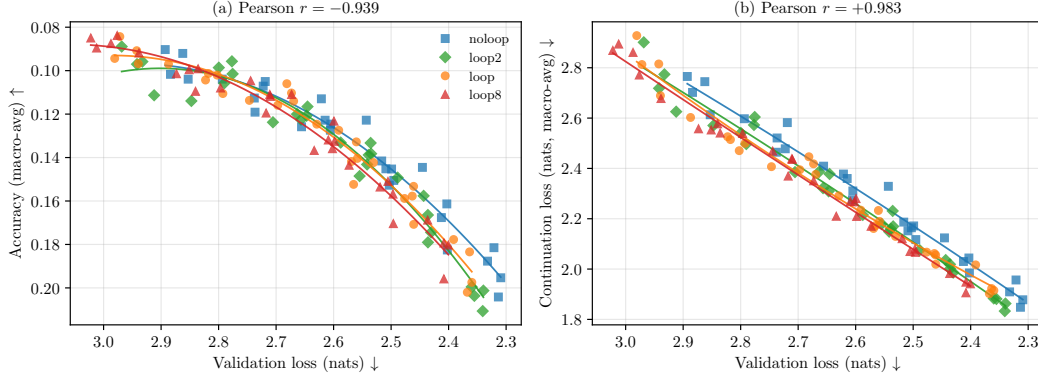


Figure 13: Macro-aggregate downstream metric vs. validation loss across iso-FLOPs checkpoints. Left: accuracy. Right: continuation loss. The x-axis is inverted so that lower-loss (more capable) models sit to the right. The accuracy y-axis is inverted so that “better” is downward on both panels.

Table 10: Extrapolation point at $s=34$, 47B tokens. Gap is $r=4$ minus $r=1$ (positive means looped trails).

	$r=1$	$r=4$	Gap
Validation loss (nats)	2.047	2.108	+0.061
Parametric knowledge (nats)	2.585	2.693	+0.108
Reading comprehension (nats)	0.980	1.030	+0.050
Math word problems (nats)	1.494	1.522	+0.028
Reasoning primitives (nats)	0.873	0.968	+0.095
Compositional symbolic (nats)	1.546	1.542	−0.004

Table 10 reports validation loss and per-axis downstream continuation loss. The looped model trails by 0.061 nats in validation loss, inside the $[0.05, 0.08]$ nats $r=4$ band measured at the iso-FLOPs grid (Section 4.2). Downstream, the three-regime pattern of Section H.2 is preserved. Parametric knowledge retains a capacity cost, the open-book axes track validation loss, and reasoning primitives show no signal in favour of the looped model.